

AI Interview Coach: 5-Day Production Build Guide

1. Product goal

Build a production-ready AI Interview Coach that can:

- Accept a resume upload.
- Extract skills, projects, experience, and probable role fit.
- Generate a tailored interview session.
- Collect user answers through text first, then optionally voice.
- Score answers across communication, technical depth, and confidence.
- Track weaknesses and generate a personalized study plan.
- Remember previous interviews so the next session becomes better.

This project is designed as an adaptive interview preparation system rather than a generic question generator, so the key product value is personalization and memory across sessions.

2. Final product scope

Core features for launch

- Resume upload and parsing.
- Candidate profile creation.
- Interview question generation.
- Text-based interview chat.
- Answer evaluation with scores and feedback.
- Weakness tracking.
- Personalized learning roadmap.
- Session history and memory.

Nice-to-have features

- Voice interview with speech-to-text and text-to-speech.
- Role-specific interview modes.

- Company-specific question sets.
 - Coding interview mode with code execution and feedback.
 - Analytics dashboard for progress over time.
-

3. Recommended stack

Use the stack suggested in the project brief:

- Frontend: Next.js 15 App Router, TypeScript, Tailwind CSS, shadcn/ui.
 - AI UI: Vercel AI SDK for streaming responses.
 - Backend: Next.js API routes.
 - Orchestration: LangGraph or equivalent multi-agent workflow.
 - Database: MongoDB.
 - Deployment: Vercel for the app, MongoDB Atlas for data.
-

4. Architecture

Main modules

1. Resume Analyzer Agent.
2. Interview Agent.
3. Evaluation Agent.
4. Learning Agent.
5. Memory Store.
6. Orchestrator.

Request flow

- User uploads resume.
- Resume Analyzer extracts structured candidate data.
- Orchestrator decides interview mode and question path.
- Interview Agent generates the next question.
- User responds.
- Evaluation Agent scores the answer.
- Learning Agent stores weaknesses and creates study tasks.
- Memory Store saves interviews, scores, and progress for future sessions.

Data objects to store

- User profile.
 - Resume text and parsed fields.
 - Interview session.
 - Question-answer pairs.
 - Score breakdown.
 - Weakness tags.
 - Study plan items.
 - Session summary.
-

5. Database design

Create these collections in MongoDB:

users

- `id`
- `name`
- `email`
- `createdAt`
- `targetRole`
- `experienceLevel`

resumes

- `userId`
- `fileUrl`
- `rawText`
- `skills`
- `projects`
- `experience`
- `education`
- `parsedAt`

interviewSessions

- `userId`
- `resumeId`
- `role`
- `company`

- mode
- status
- startedAt
- endedAt

questions

- sessionId
- question
- category
- difficulty
- expectedSignals

answers

- sessionId
- questionId
- answerText
- scoreTechnical
- scoreCommunication
- scoreConfidence
- feedback
- weaknessTags

studyPlans

- userId
- sessionId
- focusAreas
- tasks
- priority
- dueDate

memory

- userId
- topic
- summary
- tags
- lastUpdated

6. Five-day execution plan

Day 1: Product setup and foundation

Goal

Set up the codebase, UI shell, database connection, and resume upload flow.

Tasks

1. Create a Next.js 15 app with TypeScript.
2. Add Tailwind CSS and shadcn/ui.
3. Set up environment variables.
4. Connect MongoDB Atlas.
5. Create the app layout, navigation, and dashboard shell.
6. Build the resume upload page.
7. Add file upload validation.
8. Store uploaded file metadata in the database.

Deliverables by end of Day 1

- Working project skeleton.
- Resume upload UI.
- Database connectivity.
- Basic user dashboard.

Acceptance criteria

- User can upload a PDF resume.
 - File is saved correctly.
 - App does not crash on refresh.
 - Environment config works in production and development.
-

Day 2: Resume parsing and profile generation

Goal

Turn raw resume text into structured candidate data.

Tasks

1. Extract text from uploaded PDF.
2. Send resume text to the AI parser.
3. Convert resume into structured JSON.
4. Extract:
 - Skills
 - Projects
 - Experience
 - Education
 - Tools and technologies
 - Target role hints
5. Save parsed output to MongoDB.
6. Create a profile summary UI.
7. Let the user review and edit parsed data.

Important implementation idea

Do not rely only on free-form AI output. Force a JSON schema for the resume parser so the data stays consistent and usable later.

Deliverables by end of Day 2

- Resume upload + parsing pipeline.
- Profile summary page.
- Structured candidate record.

Acceptance criteria

- Resume parsing is repeatable.
 - JSON output is valid.
 - User can see extracted skills and experience.
-

Day 3: Interview generation and live interview flow

Goal

Build the actual interview experience.

Tasks

1. Create interview start form.
2. Ask user for:
 - Target role
 - Company name
 - Experience level
 - Interview type
3. Build the interview session page.
4. Generate questions from resume + target role.
5. Stream questions one by one.
6. Store each question in the database.
7. Save user answers in real time.
8. Add “next question” logic.
9. Support categories such as:
 - Behavioral
 - Technical
 - Resume-based
 - Project-based
 - Role-specific

Question-generation logic

Use a rule-based structure before the LLM prompt:

- Start with resume-based questions.
- Then ask role-specific technical questions.
- Then ask behavioral questions.
- Finish with weakness-focused follow-ups.

Deliverables by end of Day 3

- Full interview session UI.
- Dynamic question generation.
- Answer capture and session persistence.

Acceptance criteria

- Interview can run from start to finish.
 - Questions feel personalized.
 - User answers are saved correctly.
-

Day 4: Evaluation, feedback, and learning plan

Goal

Turn interview answers into useful feedback.

Tasks

1. Build evaluation prompt and scoring schema.
2. Score each answer on:
 - Technical depth
 - Communication clarity
 - Confidence
 - Structure
 - Relevance
3. Generate written feedback.
4. Detect weakness patterns.
5. Aggregate session-level insights.
6. Build a learning plan generator.
7. Save focus areas into memory.
8. Add summary dashboard:
 - Average scores
 - Weakest topics
 - Progress over time
 - Recommended study items

Feedback format

For each answer, show:

- Score
- What was strong
- What was weak
- What to improve next time
- Example follow-up practice topic

Deliverables by end of Day 4

- Answer evaluation engine.
- Personalized learning roadmap.
- Interview summary dashboard.

Acceptance criteria

- Every answer gets structured feedback.
 - Weaknesses are saved and reused later.
 - User sees a practical next-step plan.
-

Day 5: Production hardening and launch

Goal

Make the project stable, polished, and deployable.

Tasks

1. Add authentication if needed.
2. Add loading states and error handling.
3. Add retry logic for AI failures.
4. Validate all API inputs.
5. Add rate limiting for AI endpoints.
6. Sanitize uploaded files.
7. Store sensitive keys in environment variables only.
8. Test on multiple screen sizes.
9. Improve copy, spacing, and UX polish.
10. Deploy to Vercel.
11. Configure MongoDB Atlas production access.
12. Run final end-to-end testing.
13. Prepare launch page and demo flow.

Deliverables by end of Day 5

- Production deployment.
- Stable, testable product.
- Clean UI and onboarding flow.

- Ready-to-demo interview coach.
-

7. AI prompt design

Resume parser prompt

Make the model return strict JSON only.

Interview generator prompt

Inputs:

- Resume summary
- Target role
- Experience level
- Previous weakness tags
- Interview stage

Output:

- Next question
- Question type
- Difficulty
- Why this question matters
- Expected good answer signals

Evaluation prompt

Inputs:

- Question
- User answer
- Candidate profile
- Expected answer traits

Output:

- Score breakdown
- Feedback
- Mistakes
- Improvement suggestion
- Weakness tags

Learning-plan prompt

Inputs:

- Weakness tags
- Interview scores
- Role goals

Output:

- Study topics
 - Practice tasks
 - Priority order
 - Estimated time needed
-

8. UX flow

Onboarding

1. User lands on app.
2. Uploads resume.
3. Selects target role.
4. Chooses interview mode.
5. Starts session.

Interview screen

- Shows current question.
- Lets user type answer.
- Can optionally record voice later.
- Shows timer if needed.
- Saves answer automatically.

Results screen

- Shows score cards.
 - Shows weaknesses.
 - Shows recommended practice.
 - Offers a retry interview.
-

9. Production quality checklist

Before launch, verify:

- Resume upload works with valid PDFs.
 - Invalid files are rejected.
 - AI outputs are schema-validated.
 - Database records are created correctly.
 - Session resume/reload works.
 - Answer evaluation is consistent.
 - UI is responsive on mobile and desktop.
 - API keys are protected.
 - Error messages are user-friendly.
 - Logging exists for failures.
 - Deployment is smoke-tested.
-

10. Suggested file structure

- `app/`
 - `components/`
 - `lib/`
 - `lib/mongodb/`
 - `lib/ai/`
 - `lib/parsers/`
 - `lib/evaluation/`
 - `lib/memory/`
 - `app/api/upload/`
 - `app/api/resume-parse/`
 - `app/api/interview/start/`
 - `app/api/interview/next/`
 - `app/api/interview/evaluate/`
 - `app/api/learning-plan/`
-

11. Build order that saves time

If time gets tight, prioritize in this order:

1. Resume upload.
2. Resume parsing.

3. Interview question generation.
4. Answer evaluation.
5. Learning plan.
6. Memory and analytics.
7. Voice features.

This order keeps the app launchable even if advanced features are unfinished.

12. Launch checklist

- Demo dataset ready.
- Test resume uploaded.
- One full interview session recorded.
- Feedback screen polished.
- Production domain connected.
- Environment variables verified.
- Analytics or logging enabled.
- README and setup notes written.
- Final bug sweep completed.